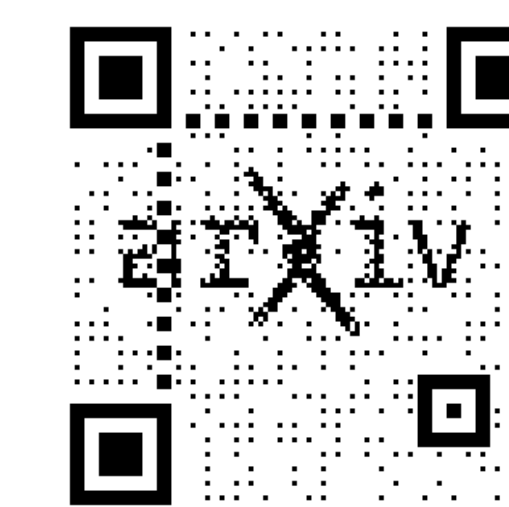




DisCa: Accelerating Video Diffusion Transformers with Distillation-Compatible Learnable Feature Caching

Chang Zou, Changlin Li, Yang Li, Patrol Li, Jianbing Wu, Xiao He, Songtao Liu, Zhao Zhong, Kailin Huang, Linfeng Zhang✉



Overview

TL;DR:

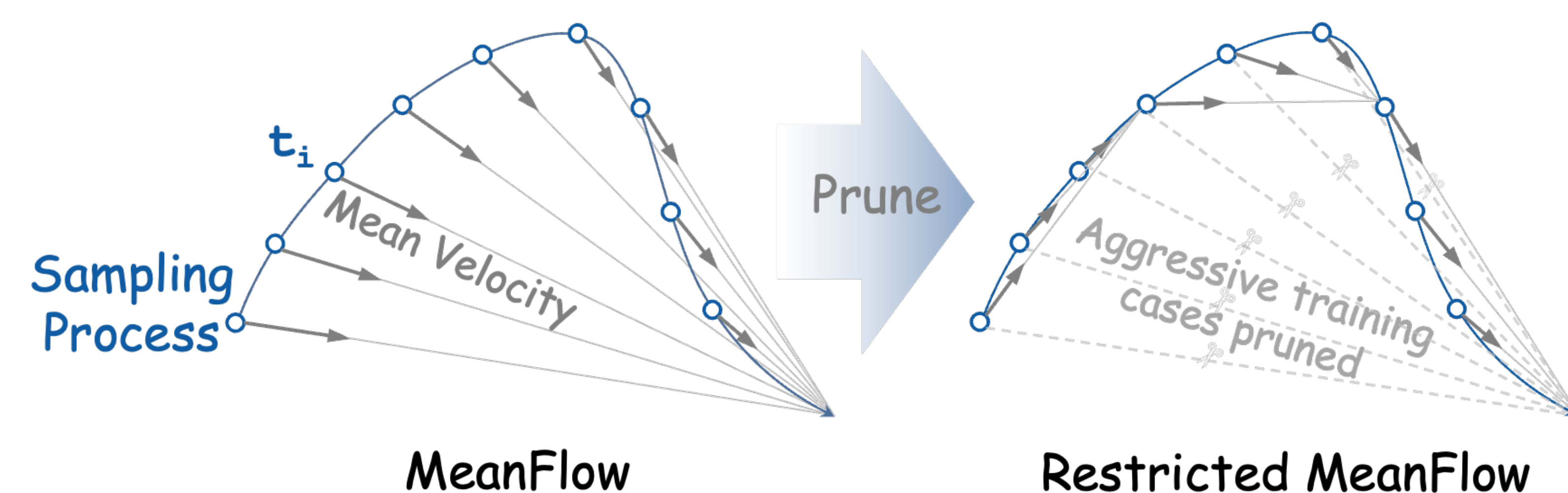
We introduce **DisCa**, a novel **Distillation-compatible learnable feature Caching** framework for **diffusion acceleration**, especially video generation.

DisCa proposes a more stable improvement to MeanFlow distillation and introduces a neural-network-based predictor on the step-distilled model, enabling further acceleration after distillation. Compared with existing methods, DisCa **better preserves spatial structures and high-frequency details**. Related techniques have been applied to HunyuanVideo 1.5.

Contributions

- propose an **more stable variant of MeanFlow distillation** by simply pruning the highly-compressed training scenarios.
- introduce a **lightweight learnable NN predictor for feature caching on step-distilled models**, better exploiting the high-dimensional features of diffusion models compared with polynomial-based prediction schemes (e.g. TaylorSeer), enabling **more than 10x acceleration** with high quality.
- More stable spatial structures, better high-frequency details.**

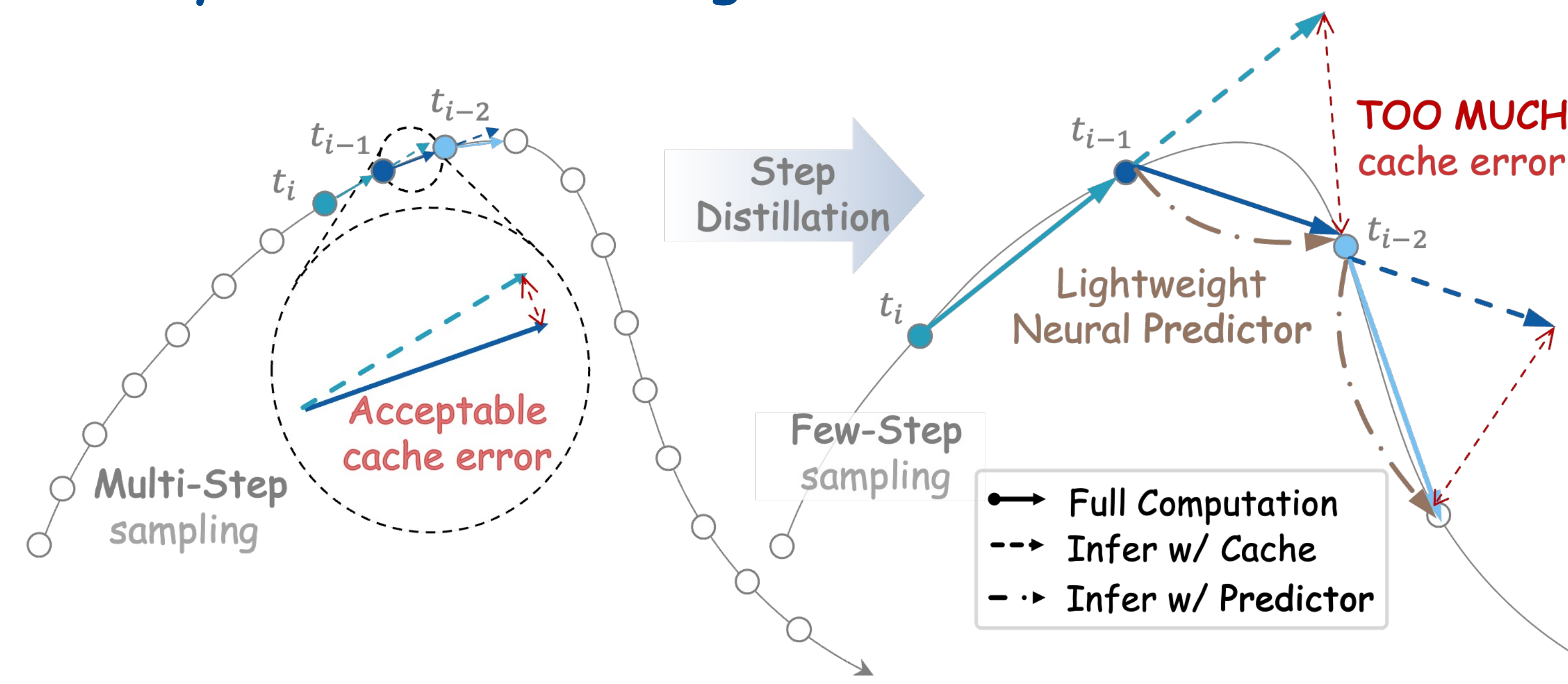
Improve MeanFlow Distillation with Simply Pruning



Extremely Compressed Cases Harms Training: MeanFlow was originally designed for one-step generation. However, inference returns to a few-step regime to preserve generation quality. **Those extremely compressed cases are not used during inference, yet negatively affect generation quality.**

Stabilizing MeanFlow Training/Distillation: To improve the stability, we **prune extreme training cases** by constraining the start-end timestep interval to a narrower range, instead of sampling it randomly from the full $[0, 1]$ range.

Key Idea: Introducing Learnable Feature Predictor

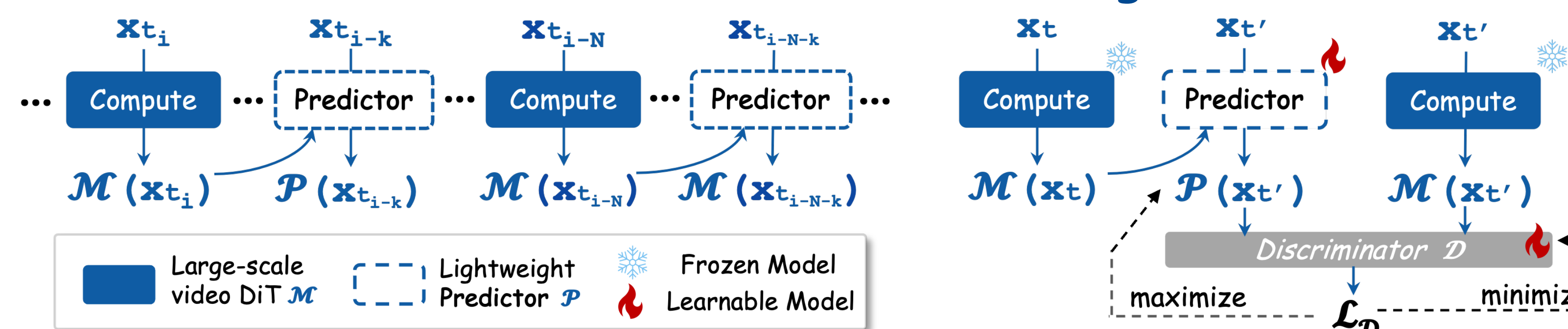


(a) Feature Caching w/o step-distillation (b) Feature Caching w/ step-distillation

Traditional Caching Fails in Few-Step Models: Traditional Caching highly relies on the inter-step feature similarity, which significantly decreases in few-step models.

Introduce Learnable Predictor: We introduce a **lightweight Neural predictor** that **better exploits the implicit information in high-dimensional features to make precise predictions**, making it possible to gain extra acceleration on step-distilled models.

DisCa: Inference & Training



(a) Inference with the Predictor

(b) Training of the Predictor

Inference: Light neural predictor $\mathcal{P}$ utilize the previous features from the large main diffusion model $\mathcal{M}$, and the noisy input $\mathbf{x}$ to predict the velocity at current step. Example sequence: $\mathbf{x}_1(\text{noise}) \rightarrow \mathcal{M} \rightarrow \mathcal{P} \rightarrow \mathcal{M} \rightarrow \mathcal{P} \rightarrow \mathcal{P} \rightarrow \mathcal{M} \rightarrow \mathcal{P} \rightarrow \mathcal{M} \rightarrow \mathbf{x}_0(\text{output})$

Adversarial Training for Predictor: We introduce GAN to train the predictor. The lightweight predictor $\mathcal{P}$ learns to approximate the outputs of the large-scale diffusion model $\mathcal{M}$ using previous features $\mathcal{M}(\mathbf{x}_t)$, while the discriminator learns to distinguish predicted features $\mathcal{P}(\mathbf{x}_{t'})$ from backbone-produced features $\mathcal{M}(\mathbf{x}_{t'})$.

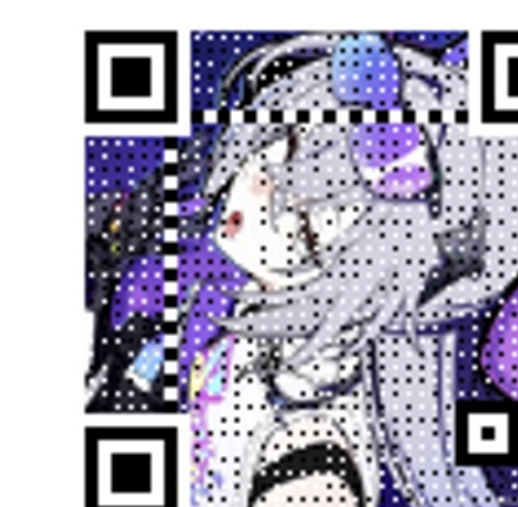
Comment: it seems this can also be viewed as a variant of on-policy distillation.

Quantitative and Qualitative Results

Method	CFG Distilled	Efficiency			VBench Score (%)		
		Latency(s) ↓	Speed ↑	Peak VRAM ↓	Semantic ↑	Quality ↑	Total ↑
HunyuanVideo 1.0 [T2V] [62]							
Original: 50 steps	✗	1155.3	1.00×	99.23GB	73.5(+0.0%)	81.5(+0.0%)	79.9(+0.0%)
CFG Distilled: 50 steps	✓	581.1	1.99×	97.21GB	66.7(-9.3%)	80.6(-1.1%)	77.9(-2.5%)
CFG Distilled: 10 steps	✓	119.7	9.65×	97.21GB	59.0(-19.7%)	76.8(-4.7%)	73.2(-8.4%)
Δ -DiT($N=8$) [7]	✓	253.7	4.55×	97.68GB	42.7(-41.9%)	70.9(-13.0%)	65.2(-18.4%)
PAB($N=8$) [76]	✓	178.8	6.46×	121.3GB	56.3(-23.4%)	76.1(-6.6%)	72.1(-9.8%)
TeaCache($l=0.4$) [31]	✓	125.3	9.22×	97.70GB	62.1(-15.5%)	78.7(-3.4%)	75.4(-5.6%)
FORA($N=6$) [56]	✓	144.2	8.01×	124.6GB	57.5(-21.8%)	76.4(-6.3%)	72.6(-9.1%)
TaylorSeer($N=6, O=1$) [32]	✓	166.0	6.96×	130.7GB	63.7(-13.3%)	79.9(-2.0%)	76.7(-4.0%)
Restricted MeanFlow: 9 steps[Ours]	✓	108.3	10.7×	97.21GB	67.8(-7.8%)	81.0(-0.6%)	78.4(-1.9%)
DisCa($\mathcal{R}=0.2, N=3$) [Ours]	✓	130.7	8.84×	97.64GB	70.3(-4.4%)	81.8(+0.4%)	79.5(-0.5%)
DisCa($\mathcal{R}=0.2, N=4$) [Ours]	✓	97.7	11.8×	97.64GB	69.3(-5.7%)	81.1(-0.5%)	78.8(-1.4%)

Method	NFE↓	Latency(s)↓	LPIPS↓	PSNR↑	SSIM↑
HunyuanVideo 1.5 [I2V]	50	778.1	-	-	-
TaylorSeer	8	123.1	0.2050	19.34	0.7137
MagCache	8	127.6	0.2093	19.03	0.7056
AdaCache	8	129.4	0.2084	19.08	0.7070
DCM	8	124.5	0.2496	18.04	0.6616
R-MeanFlow($R=0.2$)	8	124.5	0.1778	19.85	0.7186
TaylorSeer	4	79.6	0.2470	18.74	0.6996
MagCache	4	64.5	0.2416	18.73	0.7009
AdaCache	4	64.4	0.2551	18.40	0.6889
DCM	4	62.3	0.2450	17.84	0.6686
DisCa($N=2, R=0.2$)	4	64.2	0.1942	19.29	0.7071

Method	VBench Score (%)		
	Semantic ↑	Quality ↑	Total ↑
HunyuanVideo 1.0 [T2V] [62]			
Original: 50 steps	73.5(+0.0%)	81.5(+0.0%)	79.9(+0.0%)
CFG Distilled: 50 steps	66.7(-9.3%)	80.6(-1.1%)	77.9(-2.5%)
MeanFlow: 20 steps	66.6(+0.0%)	81.8(+0.0%)	78.8(+0.0%)
Restricted MeanFlow($\mathcal{R}=0.4$)	70.2(+4.5%)	82.0(+0.2%)	79.7(+1.1%)
Restricted MeanFlow($\mathcal{R}=0.2$)	70.4(+5.7%)	81.8(+0.0%)	79.5(+0.9%)
MeanFlow: 10 steps	60.9(+0.0%)	80.6(+0.0%)	76.7(+0.0%)
Restricted MeanFlow($\mathcal{R}=0.4$)	67.6(+11.0%)	81.3(+0.9%)	78.6(+2.5%)
Restricted MeanFlow($\mathcal{R}=0.2$)	68.2(+12.0%)	81.3(+0.9%)	78.7(+2.9%)

QR Code for
Videos Cases!